

Toward Satisfiability Modulo Realizability

Andrew Krapivin , Benjamin Przyboc , and Marijn J. H. Heule 

Carnegie Mellon University, Pittsburgh, USA
{akrapivi,bprzyboc,mheule}@andrew.cmu.edu

Abstract. Problems complete for the existential theory of the reals ($\exists\mathbb{R}$) arise throughout discrete geometry. We introduce *satisfiability modulo realizability*, a SAT-based approach for solving satisfiable instances of $\exists\mathbb{R}$ whose solutions correspond to realizable geometric configurations. Our method encodes an underapproximation of a geometric problem as a SAT instance over abstract order types. Since almost all abstract order types are unrealizable, naive search is infeasible. We guide the search toward realizable order types using diversity-driven sampling, partial realizability feedback, and a novel flippability heuristic that passes only limited information between components. We apply our method to discrete geometry problems and resolve an open problem by showing that the largest set of points avoiding empty convex hexagons and convex heptagons is of size 23.

Keywords: Discrete geometry · SAT solving · Realizability.

1 Introduction

The success of SAT solving has inspired the motto that NP is the new P, but the same cannot yet be said for classes beyond NP. In this paper, we focus on problems that are complete for the class $\exists\mathbb{R}$ (pronounced “existential theory of the reals”), which is a continuous analogue of NP that often arises in areas including discrete geometry, game theory, and computer algebra [33].

We develop a SAT-based program, which we call PointSAT, for finding solutions to problems in discrete geometry, where solutions consist of sets of points in $\mathbb{R}^2$ satisfying combinatorial constraints.¹ We evaluate PointSAT by applying it to variants of the happy ending problem posed by Esther Klein and studied by Erdős and Szekeres [9], a problem so named because it led to Klein and Szekeres getting married. Given a set of points in $\mathbb{R}^2$ in *general position* (i.e., no three points are collinear), we say that a *k-gon* is a subset of *k* points in convex position, and a *k-hole* is a *k-gon* with no points in its interior. Using PointSAT, we obtained the following answer to a question of Heule and Scheucher [22], who proved the upper bound, with Figure 1 showing a witnessing point set:

Theorem 1. *The largest point-set in $\mathbb{R}^2$ with no 6-hole or 7-gon is of size 23.*

In addition to proving Theorem 1, we demonstrate the versatility of PointSAT by using it to find constructions for two related problems.

¹ PointSAT is available at <https://github.com/andrewkrapivin/PointSAT>.

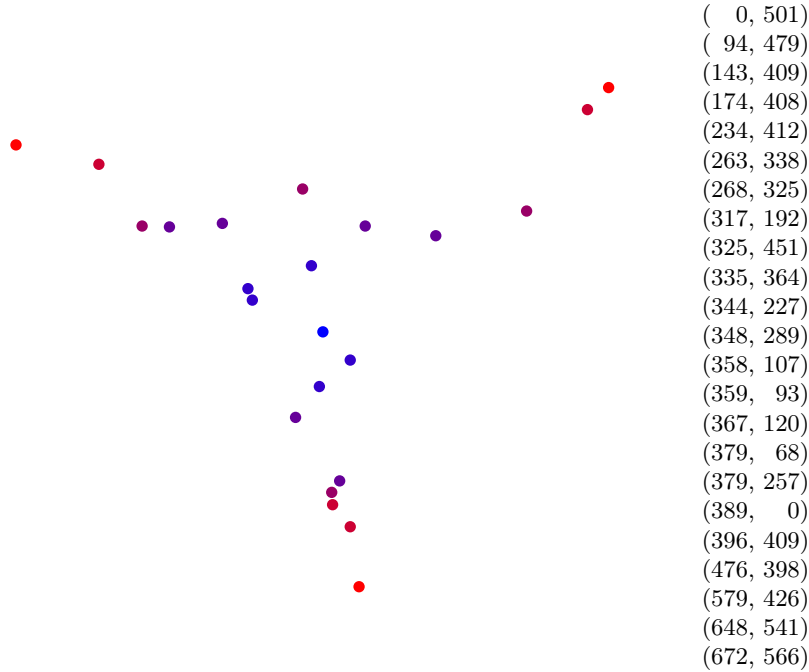


Fig. 1: A set of 23 points with no 6-hole or 7-gon

At a high level, our approach involves encoding an underapproximation of a given geometric problem into SAT, which means that every solution to the underlying geometric problem corresponds to a satisfying assignment but not vice versa. A satisfying assignment is called an *abstract order type*, and we say that an assignment corresponding to a geometric solution is *realizable*. There are two challenges to overcome:

1. Testing whether an abstract order type is realizable is computationally difficult (specifically, $\exists\mathbb{R}$ -complete).
2. Almost all abstract order types are not realizable.

Subercaseaux, Mackey, Qian, and Heule [36] recently made significant progress on the first challenge by developing **Localizer**, a local-search solver for testing the realizability of abstract order types. Given an abstract order type as input, **Localizer** initializes a point configuration and iteratively perturbs the points using a simulated annealing strategy to minimize the number of violated orientation constraints. For realizable order types, **Localizer** succeeds in finding a realization with sufficiently high frequency to make it the current state-of-the-art tool for this task.

Our contribution is to the second challenge. If almost all abstract order types are not realizable, we need some way to guide our search toward the few that

are. Indeed, while searching for a set of 23 points with no 6-hole or 7-gon, Heule and Scheucher [22] report that none of the abstract order types they tested were realizable. **PointSAT** interfaces between a SAT solver and **Localizer** while incorporating heuristics we discovered that make finding realizable order types more likely. By analogy with solvers for satisfiability modulo theories (SMT), which interface between SAT solvers and theory solvers, we call our approach *satisfiability modulo realizability*, and **PointSAT** can be seen as an SMT solver for a fragment of the existential theory of the reals, albeit one designed only for satisfiable instances.

Our paradigm differs from the previous SMT-style solvers in a few ways. First, rather than add unrealizability lemmas to the SAT instance, we instruct the SAT solver to generate a diverse set of candidate solutions to avoid getting stuck in problematic regions of the search space (see Section 4.1). Second, when the candidate solution is rejected by the “theory solver” (i.e., **Localizer**), the latter sends a nearby candidate solution to the SAT solver, which often turns out to be a solution (see Section 4.2). Third, we use a novel “flippability” heuristic, in which only partial information from a candidate solution is passed from the SAT solver to **Localizer** (see Section 4.3). This insight is based on a surprising experimental observation about solutions to geometric problems of the sort **PointSAT** applies to. All three of these heuristics are essential for **PointSAT**’s performance and capabilities.

SAT solvers have been successfully applied to many problems in pure mathematics (see, e.g., [4, 14, 20, 21, 27, 35]), and they are now an indispensable tool for computer-assisted mathematical discovery. On the other hand, since SAT solvers are not expressive enough for many mathematical problems, further progress in computer-assisted mathematics could be spurred by efficient solvers for more expressive theories. By using an SMT-style solver to resolve an open problem in pure mathematics, our work demonstrates the potential of this direction.

The outline of this paper is as follows. In Section 2, we provide context for the problems we study and discuss related work. In Section 3, we give the necessary background on the concepts from discrete geometry we employ. In Section 4, we describe how **PointSAT** works and discuss the heuristics we discovered that are essential to overcoming the second challenge mentioned above. In Section 5, we evaluate the effect these heuristics have on the performance of the solver and describe how we found the point set depicted in Figure 1. In Section 6, we apply **PointSAT** to three other problems in discrete geometry and present comparative data from these experiments. Finally, in Section 8, we conclude and discuss potential future work.

2 The Happy Ending Problem and Its Variants

In one of the first papers on Ramsey theory, Erdős and Szekeres [9] proved that for every $k \in \mathbb{N}$, there is a $g(k) \in \mathbb{N}$ such that every set of at least $g(k)$ points in $\mathbb{R}^2$ in general position contains a k -gon. In fact, they conjectured that $g(k) = 2^{k-2} + 1$; it is known that $g(k) \geq 2^{k-2} + 1$ [10]. This conjecture remains

unproven, but it is known to be true for $k \leq 6$; the case $k = 6$ was proven using a computer by Szekeres and Peters [37], and SAT solvers can now prove it in seconds [22].

A variant of the problem posed by Erdős [11] asks about avoiding k -holes. Let $h(k)$ be the minimum integer such that every set of at least $h(k)$ points in general position contains a k -hole, or let $h(k) = \infty$ if there is no such integer. The values of $h(k)$ are now fully determined: $h(3) = 3$, $h(4) = 5$, $h(5) = 10$ [19], $h(6) = 30$ [22, 31], and $h(7) = \infty$ [23]. We can also ask about simultaneously avoiding k -holes and ℓ -gons. Let $h(k, \ell)$ be the minimum integer such that every set of at least $h(k, \ell)$ points in general position contains a k -hole or an ℓ -gon. Previous constructions show that $h(5, 6) = h(5) = 10$ and $h(6, 8) = h(6) = 30$, which leaves $h(6, 7)$ as the only nontrivial remaining case for $k \leq 6$. Heule and Scheucher [22] proved that $h(6, 7) \leq 24$. Theorem 1, proven using PointSAT, states that $h(6, 7) = 24$.

Let a k -cap be a sequence of points $p_1, \dots, p_k$ with increasing x -coordinates such that p_{i+2} lies below the line from p_i to p_{i+1} for all $i \in [k - 2]$. PointSAT can find a set of 26 points with no 5-cap or 7-gon. The existence of such a set follows from a theorem of Erdős, Tuza, and Valtr [12], although PointSAT works without relying on any problem-specific knowledge. PointSAT can also find a set of 29 points with no 6-hole, replicating a result of Overmars [31]. Overmars wrote a dedicated program to search for such a set of points, which relies on a specialized algorithm for finding holes [7];² PointSAT uses no problem-specific algorithms.

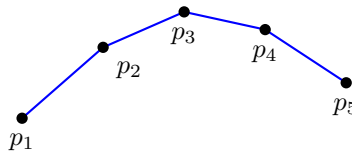


Fig. 2: A 5-cap

3 Order Types and Realizability

Problems in discrete geometry are naturally formulated as satisfiability problems with constraints involving real numbers representing the coordinates of points. However, this formulation obscures the combinatorial content of the problems, and applying an off-the-shelf solver for the existential theory of the reals to such an encoding would be hopelessly inefficient. We instead adopt a different approach based on *order types* [26], which is standard when applying SAT solvers to problems in discrete geometry (see, e.g., [22, 36]).

We consider problems concerning points in $\mathbb{R}^2$ in general position. Recall that this means that no three points are collinear. Given such a set of n points, we can rotate the set so that no two points share the same x -coordinate. Then, we can label the points from left to right with labels $1, \dots, n$. For each triple of points (i, j, k) with $1 \leq i < j < k \leq n$, let $\sigma(i, j, k) = +$ if i, j , and k are oriented counterclockwise and $\sigma(i, j, k) = -$ if they are oriented clockwise; since

² Overmars' program was publicly released, but it appears to now be lost unfortunately.

the points are in general position, i , j , and k must be oriented either clockwise or counterclockwise. This function σ is called the *order type* of the set of points, and a constraint of the form $\sigma(i, j, k) = \pm$ is called an *orientation constraint*. By considering order types rather than concrete sets of points in $\mathbb{R}^2$, we lose information about the exact positioning of the points, but we retain exactly the information we need to define the geometric notions we care about, like convexity.

Not every function $\sigma : \binom{[n]}{3} \rightarrow \{-, +\}$ corresponds to a geometric configuration of points;³ we say that such a function is a *realizable order type* if it is induced by a set of points in $\mathbb{R}^2$. Given points i , j , k , and ℓ such that $1 \leq i < j < k < \ell \leq n$, it is not hard to see that the following constraints hold for any realizable order type:

$$\begin{aligned} \sigma(i, j, k) = + \wedge \sigma(i, k, \ell) = + &\rightarrow \sigma(i, j, \ell) = + \\ \sigma(i, j, k) = - \wedge \sigma(i, k, \ell) = - &\rightarrow \sigma(i, j, \ell) = - \\ \sigma(i, j, k) = + \wedge \sigma(j, k, \ell) = + &\rightarrow \sigma(i, k, \ell) = + \\ \sigma(i, j, k) = - \wedge \sigma(j, k, \ell) = - &\rightarrow \sigma(i, k, \ell) = -. \end{aligned}$$

We call these constraints the *order type axioms*,⁴ and we call a function $\sigma : \binom{[n]}{3} \rightarrow \{-, +\}$ satisfying these constraints an *abstract order type*. The order type axioms can be naturally encoded into SAT using a boolean variable for each $\sigma(i, j, k)$ (called an *orientation variable*), making them particularly suitable for our purposes. However, not every abstract order type is realizable, and it is $\exists\mathbb{R}$ -complete to determine whether an abstract order type is realizable [29, 34]. Therefore, a compact SAT encoding of the exact constraints for realizable order types would imply that $\text{NP} = \exists\mathbb{R}$, which is considered unlikely. We don't attempt to enforce any additional realizability constraints at the SAT level.⁵

The papers [22, 36] describe how various geometric constraints can be expressed in terms of abstract order types and encoded into SAT. Specifically, [22] shows how to encode avoiding k -holes, ℓ -gons, and k -caps, and [36] shows how to enforce rotational symmetry and prescribe the sizes of the convex hull layers. For all of the solvable problems we consider in this paper, the SAT instances can be solved in minutes on a single core. The difficulty is in finding realizable solutions.

Despite the order type axioms being an underapproximation of the geometric constraints, the remarkable fact is that all of the theorems mentioned in Section 2 remain true when they are instead interpreted as statements about abstract order types. On the other hand, almost all abstract order types are not realizable: There are $2^{\Theta(n^2)}$ abstract order types on n points, but only $2^{\Theta(n \log n)}$ of them are realizable [17]. In the context of a geometric problem that can be expressed

³ By $\binom{[n]}{3}$, we mean the set of all 3-element subsets of $\{1, \dots, n\}$.

⁴ They are also sometimes called the *signotope axioms* [13].

⁵ One justification for this is that the smallest unrealizable abstract order types have 9 points [16], and forbidding these sub-configurations would seem to require $O(n^9)$ clauses, which is quite expensive.

as a constraint satisfaction problem on abstract order types, we say that an abstract order type that satisfies the constraints is an *abstract solution*; if an abstract solution is realizable, we say it is a *solution*.

4 Description of PointSAT

In this section, we describe the operation of PointSAT. We begin with an outline of the methodology, and then we describe three techniques we employed that are essential for making the approach feasible and performant.

At a high level, the basic strategy is as follows:

1. Encode the geometric problem into a SAT instance.
2. Find many abstract solutions using a SAT solver.
3. For each abstract solution, use *Localizer* to search for a realization.

This is effectively the same approach attempted by Heule and Scheucher [22], with the only difference being that they used a different tool for testing whether an abstract solution is realizable.⁶ Using this basic approach, we tried and failed to realize over 37,000 abstract solutions, replicating Heule and Scheucher’s attempt. It is unsurprising that this strategy fails, because almost all abstract solutions are not realizable. The three techniques we describe next are designed to address this problem.

As will be shown in Section 5, not all of these techniques are necessary to find a solution to prove Theorem 1. However, the combination of all of our techniques dramatically improves PointSAT’s performance, which allowed us to find over a thousand solutions proving Theorem 1. We observed that many solutions found by PointSAT have points tightly clustered together, so much so that often only a few clusters of points can be visually distinguished.⁷ Therefore, to find a solution like the one in Figure 1, where every point can be individually distinguished, it was necessary to generate many solutions. The performance gains are also useful for applying PointSAT to harder problems.

4.1 Generating Diverse Abstract Solutions

A common approach for enumerating solutions to a SAT formula is to use a tool like *allsat-CaDiCaL* [32]. For SAT formulas with few satisfying assignments, such tools are an effective way to enumerate them all. However, for the problems we consider, the number of abstract solutions is too large to exhaustively enumerate. While we can use *allsat-CaDiCaL* to generate thousands or millions of abstract solutions, most of these abstract solutions will be very similar to each other (as measured by, e.g., Hamming distance). Thus, most of the generated abstract solutions will likely be unrealizable for the same reason.

⁶ This tool was developed by Scheucher and is not publicly available.

⁷ There are theoretical results to the effect that, in the worst case, realizations of order types may require the points to be tightly clustered together [18].

To address this problem, we generate a *diverse* set of abstract solutions. Previous researchers have considered the problem of finding a diverse set of satisfying assignments of a SAT formula [15, 30, 38], but we opt for a very simple approach that suffices for our purpose. To generate an abstract solution of a formula, we use the `scranfilize` tool [3] to randomly permute the clauses of the formula, and then we use a SAT solver (such as `Kissat`) to solve the resulting formula. The effect of permuting the clauses is that the SAT solver finds a different solution each time. The resulting solutions are diverse enough to avoid the above problem.

4.2 Testing if Partial Realizations Are Solutions

When `Localizer` fails to find a realization of a given set of constraints, it outputs the best point set it found, meaning the point set with the fewest number of violated constraints. We call such a point set a *partial realization*. Partial realizations sometimes end up being solutions anyway; that is, despite not agreeing with the abstract solution we were targeting, a partial realization may still satisfy all of the desired combinatorial constraints. Thus, when `Localizer` outputs a partial realization, we always check whether it is a solution. We do this by calculating the orientations from the partial realization, adding the corresponding unit clauses to the SAT formula, and using a SAT solver to check whether these orientations indeed satisfy the combinatorial constraints.

4.3 Omitting Flippable Orientations

Given a variable assignment $\alpha : X \rightarrow \{\perp, \top\}$ and a variable $x \in X$, let α_x be the assignment obtained by flipping the assignment to variable x ; that is, $\alpha_x(x) = \overline{\alpha(x)}$ and $\alpha_x(y) = \alpha(y)$ for all $y \in X \setminus \{x\}$. Given a SAT formula φ with variables X and a satisfying assignment α , we say that a variable $x \in X$ is *flippable* if α_x is a satisfying assignment. While analyzing the structure of abstract solutions to geometric problems, we noticed that a remarkable number of orientation variables are flippable. For example, the abstract solutions of 23 points avoiding 6-holes and 7-gons that we found have about 35.9 flippable orientation variables on average (see Figure 4a).

Motivated by this observation, we came up with the following strategy. Instead of passing a complete abstract order type to `Localizer`, we omit all of the orientation constraints corresponding to flippable variables. Naturally, after dropping these constraints, the remaining constraints are more likely to be realizable, and `Localizer` is more likely to find a realization (or find a partial realization satisfying more constraints). The goal is to remove constraints that are less critical for being a solution. The trade-off is that if `Localizer` finds a realization, it may no longer be a solution; flipping the assignment to multiple flippable variables does not necessarily result in a satisfying assignment. But, perhaps surprisingly, this happens much less often than one may expect. For example, applying this strategy to the search for a set of 23 points avoiding 6-holes

and 7-gons, we found 90 realizations satisfying the non-flippable orientations of some abstract solutions, and 81 of these were solutions (see Figure 6a).

One possible explanation for why omitting flippable orientations works so well is the following. Since almost all abstract order types are non-realizable, it is almost certain that **Localizer** is going to violate some of the orientation constraints. But not all orientation constraints are of equal importance for ensuring that the partial realization is a solution: flippable orientations are less important than non-flippable orientations. By omitting the flippable orientations entirely, we incentivize **Localizer** to violate flippable orientations rather than non-flippable orientations, making the resulting partial realizations more likely to be solutions.

4.4 Summary

In summary, our revised strategy for finding a solution to a geometric problem is as follows:

1. Encode the geometric problem into a SAT instance.
2. Find a diverse set of abstract solutions by repeatedly applying **scranfilize** to the formula and solving the resulting scrambled formula using a SAT solver.
3. For each abstract solution, identify the flippable orientations and remove them to create a partial abstract solution.
4. For each partial abstract solution, run **Localizer** with some timeout and obtain a partial realization.
5. For each partial realization, test whether it is a solution.

Step 1 is done by the user, and steps 2–5 are performed by **PointSAT**.

5 Experiments for 23 Points With No 6-Gon or 7-Hole

5.1 Evaluation

In this section, we evaluate the effect of the heuristics discussed in Section 4, using the problem of finding a set of 23 points avoiding 6-holes and 7-gons as a benchmark. In Section 6, we will demonstrate that our approach generalizes to other problems in discrete geometry as well.

To evaluate the effectiveness of our heuristics, we generated 6494 abstract solutions for this problem using **CaDiCaL**. Using these abstract solutions, we evaluated the impact of testing partial realizations (Section 4.2) and omitting flippable orientations (Section 4.3), resulting in four different strategies depending on whether each heuristic is used. In each case, **PointSAT** ran **Localizer** with a timeout of 15 seconds on a single thread for each abstract solution. The number of solutions obtained using each strategy is given in Table 1.

Notice that without either of our heuristics, we do not find any solutions. Thus, the heuristics we developed were necessary for finding a set of 23 points with no 6-hole or 7-gon. Each heuristic from Sections 4.2 and 4.3 by itself allows us to find some solutions, but the results improve dramatically when they are combined.

Table 1: The number of solutions found for each of the four strategies

	Test partial realization	Do not test partial realization
Omit flippable	42	8
Keep flippable	3	0

In addition to these results, it is also insightful to consider the number of violated constraints `Localizer` was able to obtain both when flippables are kept and omitted. These data are shown in Figure 3. As one can see, omitting flippable literals shifts the distribution of the number of violations to the left, which is to be expected since there are fewer constraints. Specifically, the mean number of violations is 20.1 when flippable orientations are kept and 9.7 when they are omitted.⁸ This is part of the explanation for why the heuristic is effective. To be sure, when flippable orientations are omitted, many of these will be implicitly violated, which is not reflected in these counts and may contribute to a partial realization’s failure to be a solution. Evidently, however, the trade-off works out such that the heuristic is effective, which can be attributed to the fact that satisfying the flippable orientations is less important than the non-flippable orientations.

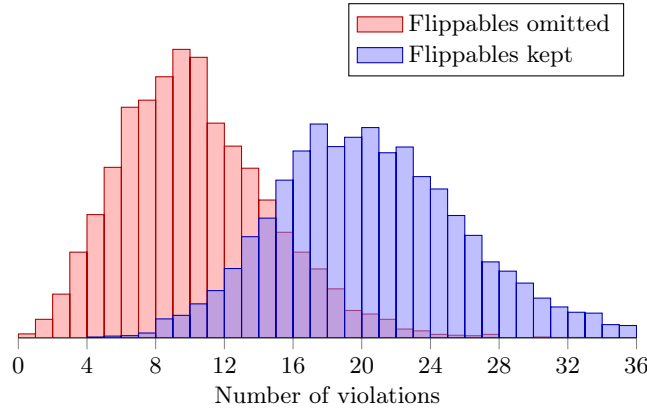


Fig. 3: Distribution of number of violations in the best partial realizations `Localizer` obtained both with and without flippable orientations

⁸ When computing statistics like these, we remove data in the top percentile, because there are rare instances where `Localizer` outputs a point set with an exceptionally high number of violations.

5.2 Finding Figure 1

Both for the sake of human-interpretability and aesthetics, we wanted to find a solution in which each of the points can be individually distinguished. Doing this required finding many solutions, selecting the best one, and applying post-processing to the point set.

We applied PointSAT to this problem by running it on the Pittsburgh Supercomputing Center [6] on a node with 128 cores for a total of 2326 core hours. It generated 174,450 abstract solutions, from which it found 1035 solutions. For each abstract solution, PointSAT ran Localizer on two threads with a 15-second timeout.

We visually inspected each solution and found only one solution for which each point was clearly individually distinguishable. To simplify the representation of the solution, we wanted to transform the solution into one for which each coordinate is a small integer.⁹ We started by dilating the point set by various factors and rounding the coordinates to integers, choosing the smallest dilation factor for which the resulting point set was still a solution. Then, we ran a script that uses simulating annealing to perturb the points with the goal of minimizing the smallest bounding box containing the points. Once this reached a local optimum, we ran a separate script that uses simulated annealing to minimize the ratio of the areas of the largest to smallest triangles. Finally, we ran the former script again, and the result was a solution that fits within the integer grid $\{0, \dots, 672\} \times \{0, \dots, 566\}$. This is the solution depicted in Figure 1.

5.3 Convex Hull Layers

Among the 174,450 abstract solutions, there were 29 different possibilities for the convex hull layers, and among the 1035 solutions, there were 17 different possibilities (see Table 2); note that the convex hull layers of an abstract order type are well defined [26]. It is interesting that $(3, 4, 4, 6, 5, 1)$ is by far the most common; the point set in Figure 1 is of this type. Every abstract solution we found has 3 points in its convex hull. It turns out that this is necessary:

Theorem 2. *Every abstract order type of 23 points with no 6-hole or 7-gon has 3 points in its convex hull.*

Proof. Our encoding enforces a symmetry breaking constraint that $\sigma(1, i, j) = +$ for all $2 \leq i < j \leq 23$ (see [22] for the justification of this constraint). Given this constraint, if there is a solution with at least 4 points in the convex hull, then there is a solution in which $\sigma(2, i, 23) = +$ for some $i \in [3, 22]$, which we can enforce with a single additional clause. We partitioned the resulting formula into cubes using the strategy from [22]; that is, we case split on all the possible assignments to a small set of orientation variables and solve the cases in parallel. The computation took 1727 core hours parallelized across 128 cores on the Pittsburgh Supercomputing Center, and each cube was unsatisfiable.

⁹ Minimizing the size of the integer grid into which a solution can be embedded is a task that has also been studied in the context of similar problems [2, 8].

Table 2: Frequency of convex hull layer sizes for 23 points with no 6-hole or 7-gon

Hull layer sizes	# abs.	# sol.	Hull layer sizes	# abs.	# sol.
(3, 3, 3, 3, 6, 5)	2	0	(3, 3, 5, 6, 5, 1)	8975	51
(3, 3, 3, 4, 4, 5, 1)	73	0	(3, 4, 3, 4, 6, 3)	621	0
(3, 3, 3, 4, 5, 4, 1)	280	0	(3, 4, 3, 5, 4, 4)	742	2
(3, 3, 3, 4, 5, 5)	128	0	(3, 4, 3, 5, 5, 3)	3576	1
(3, 3, 3, 5, 4, 5)	479	0	(3, 4, 3, 5, 6, 2)	661	2
(3, 3, 3, 5, 5, 4)	475	0	(3, 4, 3, 6, 4, 3)	6551	1
(3, 3, 3, 6, 4, 4)	5000	0	(3, 4, 4, 4, 6, 2)	5240	30
(3, 3, 3, 6, 6, 2)	134	0	(3, 4, 4, 5, 4, 3)	1600	4
(3, 3, 4, 4, 5, 4)	2	0	(3, 4, 4, 5, 5, 2)	12631	26
(3, 3, 4, 4, 6, 3)	242	1	(3, 4, 4, 6, 5, 1)	72706	556
(3, 3, 4, 5, 4, 4)	581	0	(3, 4, 5, 5, 5, 1)	3364	28
(3, 3, 4, 5, 5, 3)	1369	0	(3, 4, 5, 6, 5)	13368	61
(3, 3, 4, 5, 6, 2)	2359	11	(3, 5, 5, 5, 4, 1)	6023	63
(3, 3, 4, 6, 4, 3)	19807	40	(3, 5, 6, 6, 3)	6891	157
(3, 3, 4, 6, 6, 1)	570	1			

6 Comparison of Experiments for Four Problems

In addition to proving Theorem 1, we applied PointSAT to three other constructive problems in discrete geometry:

1. the existence of 26 points with no 5-cap or 7-gon;
2. the existence of 29 points with no 6-hole; and
3. the existence of 32 points with no 7-gon.

Note that these problems were known to have solutions prior to the present paper. For each problem, we ran PointSAT on the Pittsburgh Supercomputing Center on a node with 128 cores. We configured PointSAT to run Localizer on one thread with a 15-second timeout for each abstract solution. To fairly compare the performance of PointSAT on the four problems, we decided to run PointSAT again on the 23 point problem, both because our experiment in Section 5.2 ran Localizer on two threads instead of one and because we had made some changes to PointSAT in the meantime (most significantly, we changed how PointSAT generates diverse abstract solutions).¹⁰

For the 23 point problem, we ran PointSAT for 1811 core hours, and it generated 200,000 abstract solutions, from which it found 423 solutions. For the 26 point problem, we ran PointSAT for 1193 core hours, and it generated 200,000 abstract solutions, from which it found 32 solutions. For the 29 point problem,

¹⁰ In an earlier prototype of PointSAT, we generated diverse abstract solutions by partitioning the formula into cubes using the strategy from [22]. We tried to solve each cube using a SAT solver and kept the satisfying assignments from the solvable cubes. We later found that we could generate diverse abstract solutions more efficiently using scanfilize.

we ran **PointSAT** for 3433 core hours, and it generated 18,806 abstract solutions, from which it found 4 solutions. For the 32 point problem, we ran **PointSAT** for 2191 core hours, and it generated 200,000 abstract solutions but did not find any solutions. As expected, the problems become more difficult as the number of points increases, since a smaller fraction of abstract solutions are realizable.

In comparison with the other problems, finding an abstract solution to the 29 point problem is abnormally difficult; it takes on average 640 seconds to find an abstract solution with **Kissat**, while the other problems can all be solved in under 30 seconds. This is why we only generated 18,806 abstract solutions for this problem. On average, it took **PointSAT** 858 core hours to find a solution to this problem, since we found 4 solutions with 3433 core hours. Overmars [31] reports that his dedicated program, which is no longer available, could find a solution in about 4 days on a Pentium III 500 MHz, so **PointSAT** is not competitive with his program, although it is more generally applicable. This suggests that further improvements to **PointSAT** are possible.

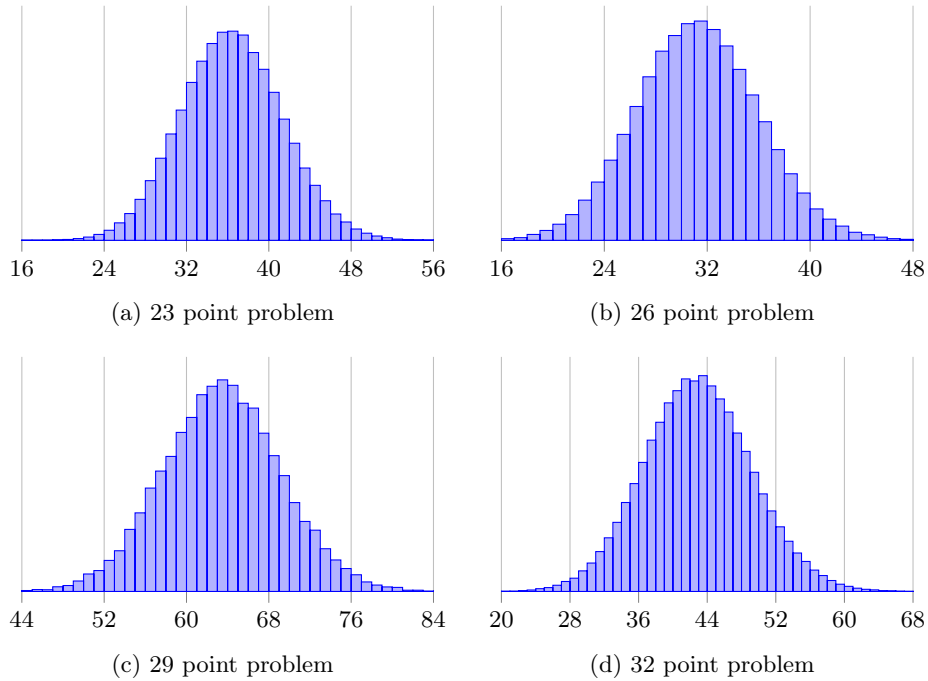


Fig. 4: Number of flippable orientations

6.1 Number of Flippable Orientations

Figure 4 shows the distributions for the number of flippable orientations in abstract solutions for each of the four problems. Each distribution is roughly normally distributed with a mean of 35.9 for the 23 point problem (2.0% of all triples), 30.7 for the 26 point problem (1.2% of all triples), 63.0 for the 29 point problem (1.7% of all triples), and 42.4 for the 32 point problem (0.9% of all triples). The ratio of the number of flippable orientations to the number of triples of points was smaller for the 32 point problem than for the other three problems, which limits the effectiveness of omitting flippable orientations. This suggests that, besides having more points, the 32 point problem may have some additional problem-specific difficulty as well.

6.2 Distribution of Number of Violations

Figure 5 shows the distributions for the number of violations for partial realizations (regardless of whether they are solutions) for each of the four problems. Each distribution is roughly normally distributed with a mean of 12.2 for the 23 point problem, 38.0 for the 26 point problem, 21.8 for the 29 point problem, and 121.6 for the 32 point problem. The data suggest that abstract solutions to the 26 and 32 point problems are more difficult to realize than abstract solutions for the 23 and 29 point problems.

6.3 Solution With a Given Number of Violations

Figures 6 and 7 show the probability of being a solution versus the number of violations and the proportion of solutions with a given number of violations. Note that we only present the data for the 23 and 26 point problems, because the other problems had too few or no solutions. The data for the 26 point problem are noisy, because there are only 32 solutions, but some patterns are still evident. There is a roughly exponential relationship between the number of violations in a partial realization and the probability that it is a solution. For the 23 point problem, the modal number of violations for the partial realizations that were solutions is 1, and for the 26 point problem, the mode is 9. This demonstrates the importance of checking whether a partial realization is a solution even when the number of violations is nonzero.

7 Related Work

Some previous work has applied SMT-style solvers to make progress on open mathematical problems. One paradigm is *SAT+CAS* [1, 39], which combines SAT solvers with a computer algebra system (CAS) to solve problems involving symbolic computation. In this approach, partial candidate solutions are passed from the SAT solver to the CAS; if the CAS detects a problem with the candidate, a theory lemma is added to the SAT instance. This approach has been

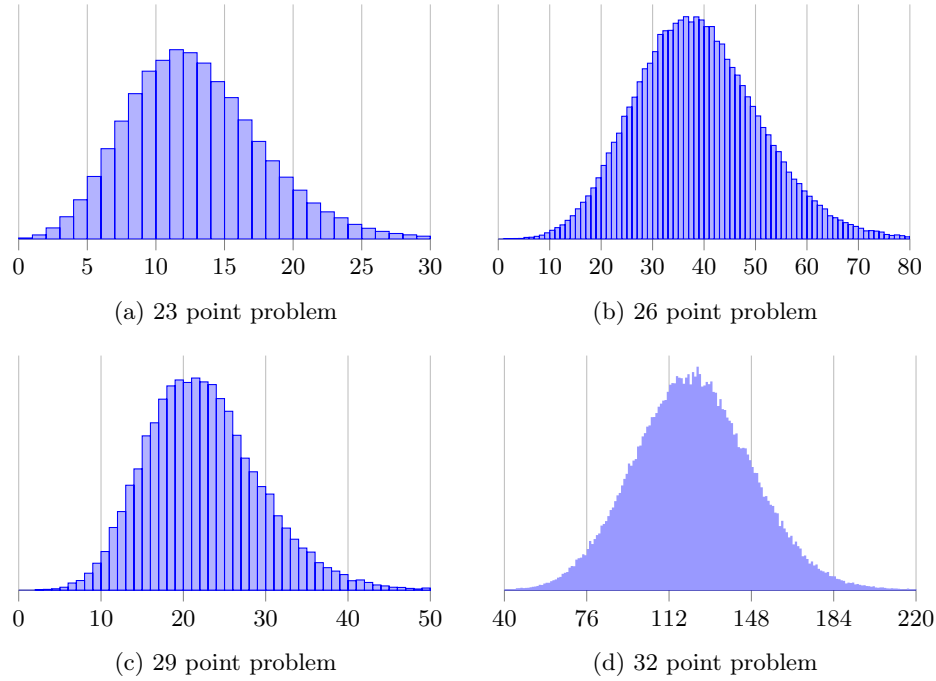


Fig. 5: Number of violations

applied to study small cases of the Williamson conjecture [5], a problem from design theory, and to improve the bounds on the minimum Kochen–Specker problem [28], a problem from quantum mechanics. Another SMT-style paradigm is *satisfiability modulo symmetries* (SMS), in which a SAT solver passes partial candidate solutions to a program that checks if the candidate is in canonical form; if not, a theory lemma is generated. This approach has been applied to prove even tighter bounds on the minimum Kochen–Specker problem [24] and verify small cases of the Murty–Simon conjecture in graph theory [25].

8 Conclusion and future work

We introduced **PointSAT**, a program for finding constructions in discrete geometry by interfacing between a SAT solver and **Localizer**. We demonstrated the effectiveness of **PointSAT** by resolving an open problem about the existence of 23 points with no 6-hole or 7-gon; in fact, we can find thousands of witnessing point sets, while a previous approach was unable to find even one. Additionally, we showed that **PointSAT** is applicable to other problems in discrete geometry, which we hope will make it a useful tool for researchers in the area. The main conceptual insight, described in Section 4.3, is that solutions to these types of problems have many flippable orientations, and that ignoring these orientations

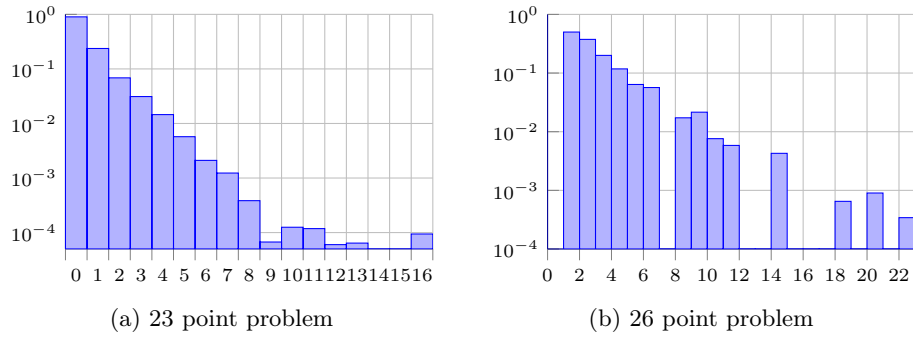


Fig. 6: Probability of being a solution versus number of violations

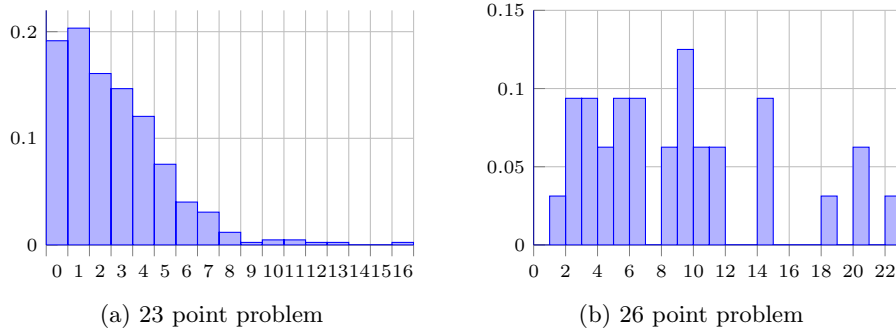


Fig. 7: Proportion of solutions with a given number of violations

when searching for a solution makes it much easier to find one. This insight has the potential to be useful for a variety of problems in discrete geometry, and its discovery is yet another success story for the application of SAT to experimental mathematics.

Despite the promising initial progress, there is still room for further improvements. For instance, **PointSAT** is still not competitive with Overmars' dedicated program for finding 29 points with no 6-hole, and it does not scale well to problems with more than 30 points. The biggest limitation of our approach seems to be that the SAT solver is still groping in the dark so to speak, finding abstract solutions without regard for how likely they are to be realizable. Figuring out how to overcome this limitation is a promising future direction.

More broadly, it may be fruitful to find other domains in which a methodology similar to the one described here could be applied. Specifically, we expect that similar approaches could be applicable to classes of problems in which there is an underapproximation that naturally encodes into SAT and a tool for testing whether a solution to the underapproximation is a true solution. Pursuing these lines of research has the potential to make new classes of mathematical problems amenable to computer-assisted discovery.

Acknowledgments. We thank Ruben Martins and Bernardo Subercaseaux for helpful discussions. The research was supported by the National Science Foundation under grant DMS-2434625. Krapivin was supported by the NSF grant CNS2504471, Packard Foundation grant 2020-71730, and the Jeanne B. and Richard F. Berdik ARCS Pittsburgh Endowed Scholar Award. Przybocki was supported by the NSF Graduate Research Fellowship Program under Grant No. DGE-2140739.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ábrahám, E.: Building bridges between symbolic computation and satisfiability checking. In: Yokoyama, K., Linton, S., Robertz, D. (eds.) *Proceedings of the 2015 ACM on International Symposium on Symbolic and Algebraic Computation, IS-SAC 2015, Bath, United Kingdom, July 06 - 09, 2015*. pp. 1–6. ACM (2015), <https://doi.org/10.1145/2755996.2756636>
2. Barba, L., Duque, F., Fabila-Monroy, R., Hidalgo-Toscano, C.: Drawing the Horton set in an integer grid of minimum size. *Comput. Geom.* **63**, 10–19 (2017), <https://doi.org/10.1016/j.comgeo.2017.02.002>
3. Biere, A., Heule, M.: The effect of scrambling CNFs. In: Berre, D.L., Jarvisalo, M. (eds.) *Proceedings of Pragmatics of SAT 2015, Austin, Texas, USA, September 23, 2015 / Pragmatics of SAT 2018, Oxford, UK, July 7, 2018*. EPIc Series in Computing, vol. 59, pp. 111–126. EasyChair (2018), <https://doi.org/10.29007/9dj5>
4. Brakensiek, J., Heule, M., Mackey, J., Narváez, D.E.: The resolution of Keller’s conjecture. *J. Autom. Reason.* **66**(3), 277–300 (2022), <https://doi.org/10.1007/s10817-022-09623-5>
5. Bright, C., Kotsireas, I.S., Ganesh, V.: Applying computer algebra systems with SAT solvers to the Williamson conjecture. *J. Symb. Comput.* **100**, 187–209 (2020), <https://doi.org/10.1016/j.jsc.2019.07.024>
6. Brown, S.T., Buitrago, P., Hanna, E., Sanielevici, S., Scibek, R., Nystrom, N.A.: Bridges-2: A platform for rapidly-evolving and data intensive research. In: *Practice and Experience in Advanced Research Computing 2021: Evolution Across All Dimensions*. PEARC ’21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3437359.3465593>
7. Dobkin, D.P., Edelsbrunner, H., Overmars, M.H.: Searching for empty convex polygons. *Algorithmica* **5**(4), 561–571 (1990), <https://doi.org/10.1007/BF01840404>
8. Duque, F., Fabila-Monroy, R., Hidalgo-Toscano, C.: Point sets with small integer coordinates and no large convex polygons. *Discrete Comput. Geom.* **59**(2), 461–476 (2018), <https://doi.org/10.1007/s00454-017-9931-6>
9. Erdős, P., Szekeres, G.: A combinatorial problem in geometry. *Compositio Math.* **2**, 463–470 (1935), http://www.numdam.org/item?id=CM_1935__2__463_0
10. Erdős, P., Szekeres, G.: On some extremum problems in elementary geometry. *Ann. Univ. Sci. Budapest. Eötvös Sect. Math.* **3**/4, 53–62 (1960/61)
11. Erdős, P.: Combinatorial problems in geometry and number theory. In: *Relations between combinatorics and other parts of mathematics (Proc. Sympos. Pure Math., Ohio State Univ., Columbus, Ohio, 1978)*, *Proc. Sympos. Pure Math.*, vol. XXXIV, pp. 149–162. Amer. Math. Soc., Providence, RI (1979)

12. Erdős, P., Tuza, Z., Valtr, P.: Ramsey-remainder. *European J. Combin.* **17**(6), 519–532 (1996), <https://doi.org/10.1006/eujc.1996.0045>
13. Felsner, S., Weil, H.: Sweeps, arrangements and signotopes. vol. 109, pp. 67–94 (2001), [https://doi.org/10.1016/S0166-218X\(00\)00232-8](https://doi.org/10.1016/S0166-218X(00)00232-8), 14th European Workshop on Computational Geometry CG’98 (Barcelona)
14. Gardam, G.: A counterexample to the unit conjecture for group rings. *Ann. of Math.* (2) **194**(3), 967–979 (2021), <https://doi.org/10.4007/annals.2021.194.3.9>
15. Gomes, C.P., Sabharwal, A., Selman, B.: Near-uniform sampling of combinatorial spaces using XOR constraints. In: Schölkopf, B., Platt, J.C., Hofmann, T. (eds.) *Advances in Neural Information Processing Systems 19*, Proceedings of the Twentieth Annual Conference on Neural Information Processing Systems, Vancouver, British Columbia, Canada, December 4–7, 2006. pp. 481–488. MIT Press (2006), <https://proceedings.neurips.cc/paper/2006/hash/4110a1994471c595f7583ef1b74ba4cb-Abstract.html>
16. Goodman, J.E., Pollack, R.: Proof of Grünbaum’s conjecture on the stretchability of certain arrangements of pseudolines. *J. Combin. Theory Ser. A* **29**(3), 385–390 (1980), [https://doi.org/10.1016/0097-3165\(80\)90038-2](https://doi.org/10.1016/0097-3165(80)90038-2)
17. Goodman, J.E., Pollack, R.: Upper bounds for configurations and polytopes in $\mathbf{R}^d$. *Discrete Comput. Geom.* **1**(3), 219–227 (1986), <https://doi.org/10.1007/BF02187696>
18. Goodman, J.E., Pollack, R., Sturmfels, B.: The intrinsic spread of a configuration in $\mathbf{R}^d$. *J. Amer. Math. Soc.* **3**(3), 639–651 (1990), <https://doi.org/10.2307/1990931>
19. Harborth, H.: Konvexe Fünfecke in ebenen Punktmengen. *Elem. Math.* **33**(5), 116–118 (1978)
20. Heule, M.J.H.: Schur number five. In: McIlraith, S.A., Weinberger, K.Q. (eds.) *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2–7, 2018. pp. 6598–6606. AAAI Press (2018), <https://doi.org/10.1609/aaai.v32i1.12209>
21. Heule, M.J.H., Kullmann, O., Marek, V.W.: Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In: Creignou, N., Berre, D.L. (eds.) *Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference*, Bordeaux, France, July 5–8, 2016, *Proceedings. Lecture Notes in Computer Science*, vol. 9710, pp. 228–245. Springer (2016), https://doi.org/10.1007/978-3-319-40970-2_15
22. Heule, M.J.H., Scheucher, M.: Happy ending: An empty hexagon in every set of 30 points. In: Finkbeiner, B., Kovács, L. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 30th International Conference, TACAS 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6–11, 2024, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 14570, pp. 61–80. Springer (2024), https://doi.org/10.1007/978-3-031-57246-3_5
23. Horton, J.D.: Sets with no empty convex 7-gons. *Canad. Math. Bull.* **26**(4), 482–484 (1983), <https://doi.org/10.4153/CMB-1983-077-8>
24. Kirchweger, M., Peitl, T., Szeider, S.: Co-certificate learning with SAT modulo symmetries. In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th–25th August 2023, Macao, SAR, China*. pp. 1944–1953. ijcai.org (2023), <https://doi.org/10.24963/ijcai.2023/216>

25. Kirchweger, M., Szeider, S.: SAT modulo symmetries for graph generation and enumeration. *ACM Trans. Comput. Log.* **25**(3), 1–30 (2024), <https://doi.org/10.1145/3670405>
26. Knuth, D.E.: Axioms and hulls, *Lecture Notes in Computer Science*, vol. 606. Springer-Verlag, Berlin (1992), <https://doi.org/10.1007/3-540-55611-7>
27. Konev, B., Lisitsa, A.: A SAT attack on the Erdős discrepancy conjecture. In: Sinz, C., Egly, U. (eds.) *Theory and Applications of Satisfiability Testing - SAT 2014 - 17th International Conference, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 14-17, 2014. Proceedings. Lecture Notes in Computer Science*, vol. 8561, pp. 219–226. Springer (2014), https://doi.org/10.1007/978-3-319-09284-3_17
28. Li, Z., Bright, C., Ganesh, V.: A SAT solver + computer algebra attack on the minimum Kochen-Specker problem. In: *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*. pp. 1898–1906. *ijcai.org* (2024), <https://www.ijcai.org/proceedings/2024/210>
29. Mnëv, N.E.: The universality theorems on the classification problem of configuration varieties and convex polytopes varieties. In: *Topology and geometry—Rohlin Seminar, Lecture Notes in Math.*, vol. 1346, pp. 527–543. Springer, Berlin (1988), <https://doi.org/10.1007/BFb0082792>
30. Nadel, A.: Generating diverse solutions in SAT. In: Sakallah, K.A., Simon, L. (eds.) *Theory and Applications of Satisfiability Testing - SAT 2011 - 14th International Conference, SAT 2011, Ann Arbor, MI, USA, June 19-22, 2011. Proceedings. Lecture Notes in Computer Science*, vol. 6695, pp. 287–301. Springer (2011), https://doi.org/10.1007/978-3-642-21581-0_23
31. Overmars, M.: Finding sets of points without empty convex 6-gons. *Discrete Comput. Geom.* **29**(1), 153–158 (2003), <https://doi.org/10.1007/s00454-002-2829-x>
32. Reeves, J.: *allsat-cadical*. <https://github.com/jreeves3/allsat-cadical> (2022)
33. Schaefer, M., Cardinal, J., Miltzow, T.: The existential theory of the reals as a complexity class: A compendium (2024), <https://arxiv.org/abs/2407.18006>
34. Shor, P.W.: Stretchability of pseudolines is NP-hard. In: *Applied geometry and discrete mathematics, DIMACS Ser. Discrete Math. Theoret. Comput. Sci.*, vol. 4, pp. 531–554. Amer. Math. Soc., Providence, RI (1991), <https://doi.org/10.1090/dimacs/004/41>
35. Subercaseaux, B., Heule, M.J.H.: The packing chromatic number of the infinite square grid is 15. In: Sankaranarayanan, S., Sharygina, N. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 29th International Conference, TACAS 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Paris, France, April 22-27, 2023, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 13993, pp. 389–406. Springer (2023), https://doi.org/10.1007/978-3-031-30823-9_20
36. Subercaseaux, B., Mackey, E., Qian, L., Heule, M.: Automated symmetric constructions in discrete geometry. In: de Paiva, V., Koepke, P. (eds.) *Intelligent Computer Mathematics - 18th International Conference, CICM 2025, Brasilia, Brazil, October 6-10, 2025, Proceedings. Lecture Notes in Computer Science*, vol. 16136, pp. 29–47. Springer (2025), https://doi.org/10.1007/978-3-032-07021-0_3
37. Szekeres, G., Peters, L.: Computer solution to the 17-point Erdős-Szekeres problem. *ANZIAM J.* **48**(2), 151–164 (2006), <https://doi.org/10.1017/S144618110000300X>

38. Zheng, Z., Cherif, S., Shibasaki, R.S., Li, C.M., Zhang, J.: Exact approaches for the diverse satisfiability problem. In: Casini, G., Dundua, B., Kutsia, T. (eds.) *Logics in Artificial Intelligence - 19th European Conference, JELIA 2025, Kutaisi, Georgia, September 1-4, 2025, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 16094, pp. 207–224. Springer (2025), https://doi.org/10.1007/978-3-032-04590-4_15
39. Zulkoski, E., Ganesh, V., Czarnecki, K.: MATHCHECK: A math assistant via a combination of computer algebra systems and SAT solvers. In: Kambhampati, S. (ed.) *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*. pp. 4228–4233. IJCAI/AAAI Press (2016), <http://www.ijcai.org/Abstract/16/636>